

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA



Tecnologias Para o Desenvolvimento Web

Raimundo Marcos Gonçalves Ludgério
Professor

E-mail: raimundo.marcos@ifpb.edu.br

O QUE VIMOS NA AULA PASSADA?

- Programação assíncrona
- Promises
- Métodos .then(), .catch() e .finally()
- Escrever código assíncrono moderno com async/await.

Conteúdo programático

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA

- Fazer requisições a APIs públicas
- Receber e interpretar dados em formato JSON
- Exibir esses dados dinamicamente no HTML
- Tratar erros e estados (carregando, erro, etc)

REVISANDO...

ARRAY HELPERS (MÉTODOS DE ARRAY)

Eles permitem aplicar uma função a cada elemento (ou selecioná-los, ou “reduzi-los”) sem precisar escrever o laço for manualmente.

Eles retornam novos valores (ou um novo array), sem modificar o array original (caso o callback não o faça explicitamente).

ARRAY HELPERS (MÉTODOS DE ARRAY)

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA

```
const pessoas = [
    { nome: "Alice", idade: 28, cidade: "São Paulo" },
    { nome: "Bruno", idade: 34, cidade: "Rio de Janeiro" },
    { nome: "Carla", idade: 22, cidade: "Belo Horizonte" },
    { nome: "Diego", idade: 40, cidade: "Curitiba" },
    { nome: "Eduarda", idade: 19, cidade: "Porto Alegre" },
    { nome: "Fábio", idade: 31, cidade: "Recife" },
    { nome: "Gabriela", idade: 27, cidade: "Fortaleza" },
    { nome: "Henrique", idade: 45, cidade: "Manaus" },
    { nome: "Isabela", idade: 23, cidade: "Salvador" },
    { nome: "João", idade: 37, cidade: "Brasília" }
];
```

ARRAY HELPERS (MÉTODOS DE ARRAY)

- Para esses cenários, o JS disponibiliza alguns métodos para trabalhar com arrays
- array.**forEach(function())** -> varre todos os elementos do array
- array.**find(function())** -> Retorna o primeiro elemento que satisfaz a condição de teste.

ARRAY HELPERS (MÉTODOS DE ARRAY)

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA PARAÍBA

- array.**some(function())** -> Verifica se pelo menos um elemento satisfaz a condição
- array.**every(function())** -> Verifica se todos os elementos satisfazem a condição.

ARRAY HELPERS (MÉTODOS DE ARRAY)

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA PARAÍBA

- array.**map(function())** -> percorre cada elemento do array, aplica uma função transformadora a ele, e retorna um novo array com os resultados
- array.**filter(function())** -> percorre cada elemento do array e retorna um novo array contendo somente os elementos para os quais o teste retornou true.



FOREACH()

- Não retorna nada (undefined)
- Não modifica o array original (a menos que você modifique dentro da função)
- Não pode ser interrompido com break ou return

forEach()


```
pessoas.forEach((pessoa, indice) => {
    console.log(`A pessoa ${pessoa.nome} está na posição ${indice}`)
})
```

```
pessoas.forEach((pessoa, indice) => console.log(`${pessoa.nome} - ${indice}`))
```

Varre todos os elementos.



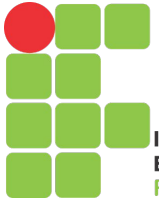
FIND()

- Retorna o elemento encontrado
- Retorna undefined se nenhum elemento satisfazer a condição
- Para no primeiro elemento que satisfaz a condição

FIND()


```
let maioridade = pessoas.find(pessoa => pessoa.idade >= 18)
console.log(maioridade)
```

```
let maioridade = pessoas.find(function (pessoa) {
    return pessoa.idade >= 18;
})
console.log(maioridade)
```



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA

some()

- Retorna boolean (true/false)
- Para na primeira ocorrência que satisfaz a condição
- Útil para validações

SOME()


```
const comecaComA = pessoas.some(pessoa => pessoa.nome.charAt(0).toUpperCase() === "A");
console.log(comecaComA) // true
```

```
const comecaComA = pessoas.some(function(pessoa) {
    return pessoa.nome.charAt(0).toUpperCase() === "A";
});
console.log(comecaComA) // true
```



EVERY()

- Retorna boolean (true/false)
- Para no primeiro elemento que não satisfaz a condição
- Array vazio retorna true (condição vacuamente verdadeira)

EVERY()


```
const comecaComA = pessoas.every(pessoa => pessoa.nome.charAt(0).toUpperCase() === "A");
console.log(comecaComA) // false
```

```
const comecaComA = pessoas.every(function(pessoa) {
    return pessoa.nome.charAt(0).toUpperCase() === "A";
});
console.log(comecaComA) // false
```



MAP()

- Percorre cada elemento do array
- Aplica uma função transformadora a ele
- Retorna um novo array com os resultados
- O array original **não é alterado** (a menos que você faça modificações explícitas nos objetos internos)
- Tamanho do novo array = tamanho do array original.

MAP()


```
const nome_pessoas = pessoas.map(function(pessoa){
    return pessoa.nome
});
console.log(nome_pessoas);
```

```
const nome_pessoas = pessoas.map(pessoa => pessoa.nome);
console.log(nome_pessoas);
```



FILTER()

- Percorre cada elemento do array
- Aplica uma função de teste (predicate) que retorna true ou false para cada elemento
- Retorna um novo array contendo somente os elementos para os quais o teste retornou true

FILTER()


```
const pessoas_de_jampa = pessoas.filter(p => p.cidade == "João Pessoa");
console.log(pessoas_de_jampa)
```

```
const pessoas_de_jampa = pessoas.filter(function (p) {
    return p.cidade == "João Pessoa"
});
console.log(pessoas_de_jampa)
```

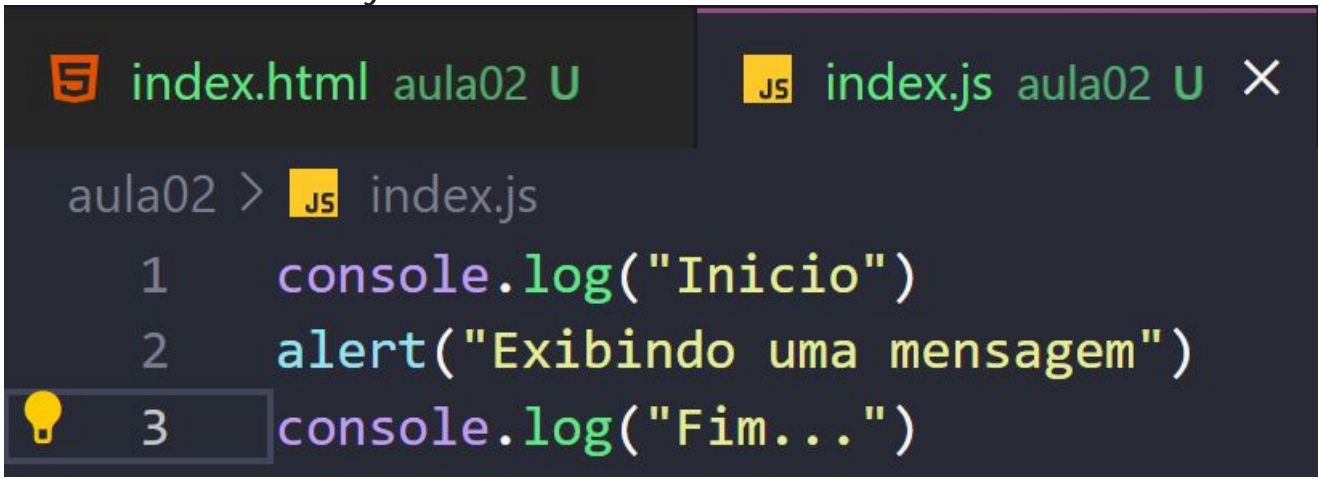
PROGRAMAÇÃO ASSÍNCRONA

PROGRAMAÇÃO ASSÍNCRONA

Um **paradigma de programação** que permite que um programa inicie tarefas potencialmente demoradas e continue executando outras operações sem esperar pela conclusão da primeira tarefa.

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA PARAÍBA

O PROBLEMA DA EXECUÇÃO SÍNCRONA



Enquanto o **alert()** está ativo, o programa para completamente.

Solução: Programação Assíncrona

Permite executar tarefas demoradas sem bloquear o fluxo principal.

Principais mecanismos:

- Callbacks
- Promises
- Async/Await

CALLBACKS

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA PARAÍBA

Callbacks

Funções **passadas como argumento** para outras funções, executadas quando uma operação é concluída.

O programador define uma função e a passa como um argumento para outra função.

Callbacks

```
function funcaoBase(callback){
    setTimeout(() => {
        callback();
    }, 2000);
};

funcaoBase(() => {
    console.log("Mensagem que aparece depois de 2s");
});
```

Callbacks - com argumentos

```
function funcaoBase(callback){
    setTimeout(() => {
        callback("Argumento passado...");
    }, 2000);
};

funcaoBase((argumento) => {
    console.log(argumento);
});
```

Callback Hell

```
fazerLogin((usuario) => {
    buscarDadosUsuario(usuario.id, (dados) => {
        carregarPosts(dados.categoria, (posts) => {
            renderizarPosts(posts, () => {
                console.log("Finalmente tudo carregou!");
            });
        });
    });
});
```

Callback Hell

```
console.log("Início do processo...");
setTimeout(() => {
    console.log("Etapa 1 concluída");
    setTimeout(() => {
        console.log("Etapa 2 concluída");
        setTimeout(() => {
            console.log("Etapa 3 concluída");
            setTimeout(() => {
                console.log("Etapa 4 concluída");
                console.log("Processo finalizado.");
            }, 1000);
        }, 1000);
    }, 1000);
}, 1000);
```

Callback Hell

```
console.log("Início do processo...");
setTimeout(() => {
    console.log("Etapa 1 concluída");
    setTimeout(() => {
        console.log("Etapa 2 concluída");
        setTimeout(() => {
            console.log("Etapa 3 concluída");
            setTimeout(() => {
                console.log("Etapa 4 concluída");
                console.log("Processo finalizado.");
            }, 1000);
        }, 1000);
    }, 1000);
}, 1000);
```



Como evitar callback hell?

- Promises
- Async/Await

PROMISES

Promises

Uma **Promise** é um objeto que representa uma operação assíncrona que pode:

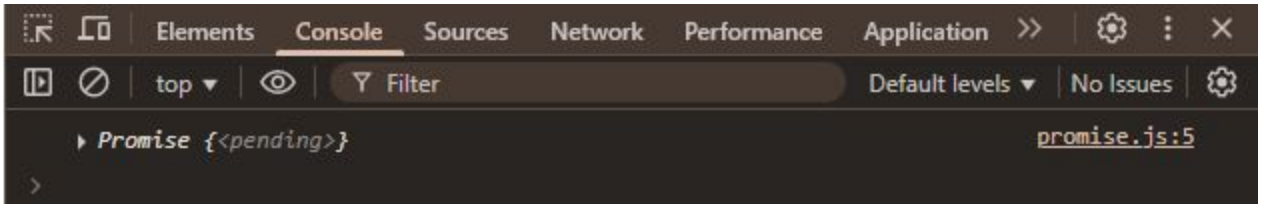
- Pendências (pending): ainda em execução
- Cumprida (fulfilled): terminou com sucesso
- Rejeitada (rejected): falho

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA PARAÍBA

Promises

```
// carregando usuários de uma API externa (exemplo)
const usuarios = fetch(
  "https://raw.githubusercontent.com/raimundoludgerio/material-didatico/
  refs/heads/main/web/javascript/aula02/usuarios.json",
);
// fetch retorna uma Promise
console.log(usuarios);
```

Promises


Elements
Console
Sources
Network
Performance
Application
top
Filter
Default levels
No Issues
Promise {<pending>}
promise.js:5

Promises

```
// carregando usuários de uma API externa (exemplo)
const usuariosPromise = fetch(
  "https://raw.githubusercontent.com/raimundoludgerio/material-didatico/
  refs/heads/main/web/javascript/aula02/usuarios.json",
);
// fetch retorna uma Promise
usuariosPromise.then(usuarios => {
    console.log(usuarios);
});
```

Promises


Elements
Console
Sources
Network
Performance
Application
top
Filter
Default levels
No Issues
promise.js:7
Response {type: 'cors', url: 'https://raw.githubusercontent.com/raimundoludgerio…fs/heads/main/web/javascript/aula02/usuarios.json', redirected: false, status: 200, ok: true, …}

Promises

O retorno da requisição retorna uma **Response** que é uma interface da **Fetch API** que representa a resposta de uma requisição de rede.

Para interpretar a resposta iremos utilizar a função **json()** que converte Response para Objeto Javascript (json).

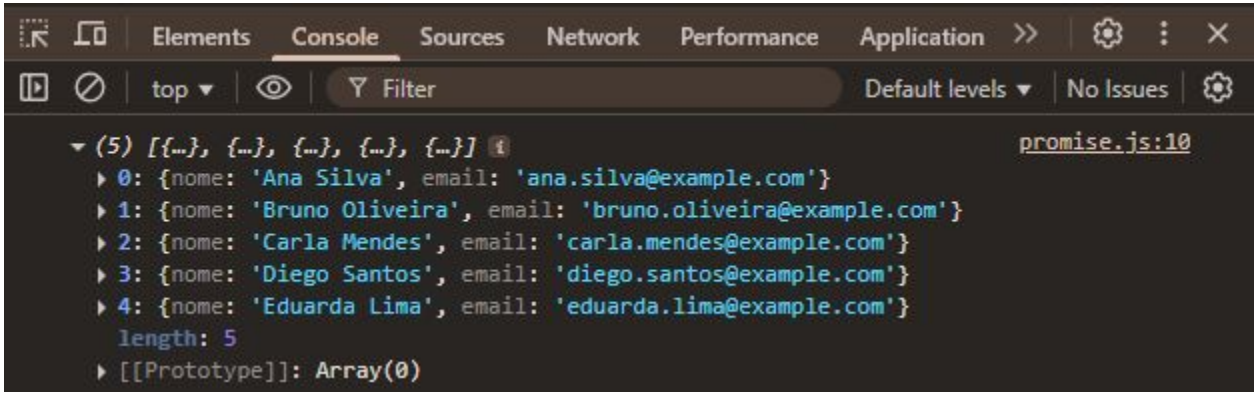
Promises

Acontece que **json()** também é assíncrono. Portanto, este é um caso em que temos que chamar duas funções assíncronas sucessivas.

Promises

```
// fetch retorna uma Promise
usuariosPromise.then(usuarios => {
    const usuariosJsonPromise = usuarios.json();
    // json também retorna uma Promise
    usuariosJsonPromise.then(usuarios => {
        console.log(usuarios);
    });
});
```

Promises


Elements
Console
Sources
Network
Performance
Application
top
Filter
Default levels
No Issues
promise.js:10
(5) [{…}, {…}, {…}, {…}, {…}]
0: {nome: 'Ana Silva', email: 'ana.silva@example.com'}
1: {nome: 'Bruno Oliveira', email: 'bruno.oliveira@example.com'}
2: {nome: 'Carla Mendes', email: 'carla.mendes@example.com'}
3: {nome: 'Diego Santos', email: 'diego.santos@example.com'}
4: {nome: 'Eduarda Lima', email: 'eduarda.lima@example.com'}
length: 5
[[Prototype]]: Array(0)

Promises

Como as Promises podem ser aninhadas, é possível encadear várias Promises, como fizemos com as Callbacks.

Como a função **then()** também retorna uma Promise, pode-se simplificar o código aninhando os retornos.

Promises

```
// Aninhando várias promises
usuariosPromise.then(response => response.json()
    .then(usuarios =>console.log(usuarios))
);
```

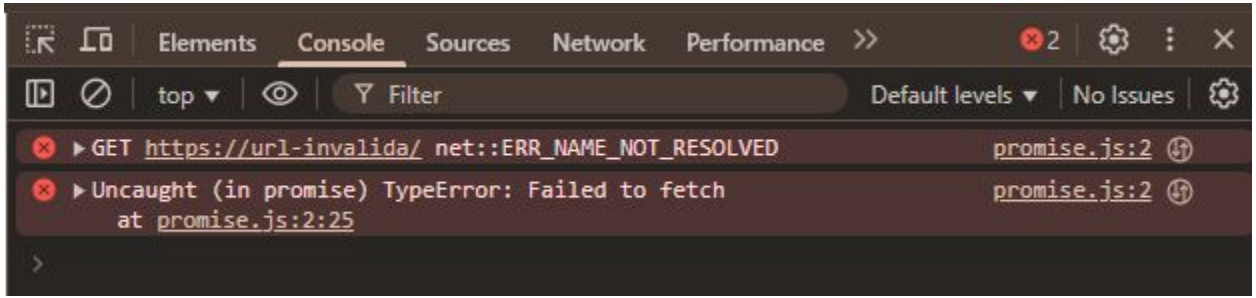
FUNÇÃO CATCH

A **API fetch()** pode lançar um erro por vários motivos (por exemplo, porque não havia conectividade de rede ou a URL estava mal formada de alguma forma) e estamos lançando um erro nós mesmos se o servidor retornar um erro.

FUNÇÃO CATCH

```
const usuariosPromise = fetch(
    "https://url-invalida",
);
// Aninhando várias promises
usuariosPromise.then(response => response.json(
    .then(usuarios =>console.log(usuarios))
);
```

FUNÇÃO CATCH



FUNÇÃO CATCH

Para suportar o **tratamento de erros**, os objetos Promise fornecem um método **catch()**.

Que é muito parecido com then(): você chama e passa uma função manipuladora.

FUNÇÃO CATCH

No entanto, enquanto o manipulador passado para **then()** é chamado quando a operação assíncrona é bem-sucedida, a função manipuladora passado para **catch()** é chamado quando a operação assíncrona falha.

FUNÇÃO CATCH

```
usuariosPromise.then(response => response.json()
    .then(usuarios =>console.log(usuarios))
).catch(error => console.log(error));
```

FUNÇÃO FINALLY

Serve para executar um **código de limpeza** ou **finalização sempre**, independentemente de a promessa ter sido resolvida com sucesso ou rejeitada por um erro.

FUNÇÃO FINALLY

É usada para realizar tarefas que precisam **acontecer sempre**, como parar um indicador de carregamento ou fechar uma conexão, antes de passar o resultado (ou erro) para o próximo manipulador, como **.then()** ou **.catch()**

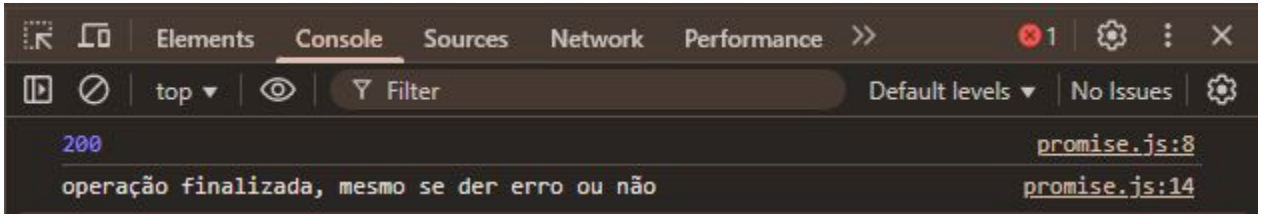
FUNÇÃO FINALLY

A função de callback fornecida ao **finally()** é executada em ambos os cenários: quando a promessa é **completada** com sucesso (resolvida) e quando ela **falha** (rejeitada).

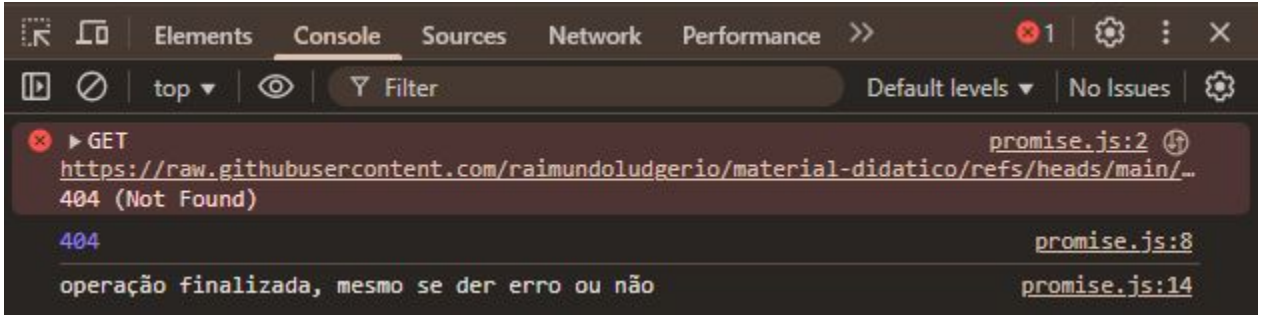
FUNÇÃO FINALLY

```
usuariosPromise.then(response => {
    // Caso de sucesso (resolve)
    console.log(response.status);
}).catch(error => {
    // Caso de sucesso (reject)
    console.log(error);
}).finally(() => {
    // Executa sempre
    console.log("operação finalizada, mesmo se der erro ou não");
});
```

FUNÇÃO FINALLY



FUNÇÃO FINALLY



CRIANDO PROMISE

Para criar uma **Promise** em JavaScript, use o construtor **new Promise()** passando uma função que recebe **resolve** e **reject** como argumentos.

CRIANDO PROMISE

Dentro dessa função executora, colocamos a lógica assíncrona, e quando o resultado desejado for obtido, chame **resolve(valor)** para o sucesso, ou **reject(erro)** para falha.

O valor passado para **resolve** é recebido no método **.then()** e o valor passado para **reject** é tratado pelo método **.catch()**.

CRIANDO PROMISE

```
const minhaPromise = new Promise((resolve, reject) => {
    // fazendo qualquer coisa assíncrona
});
```

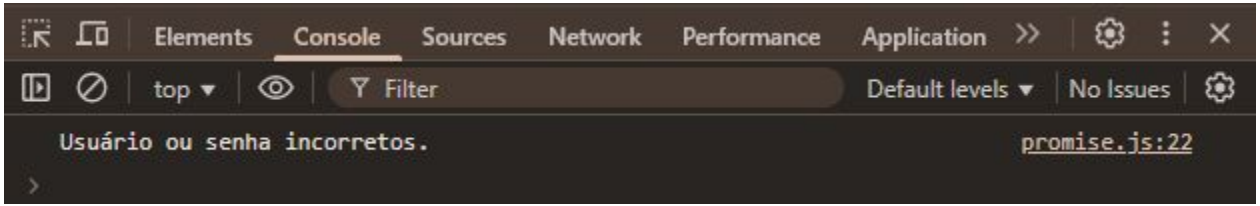
CRIANDO PROMISE

```
const minhaPromise = (usuario, senha) => new Promise((resolve, reject) => {
    setTimeout(() => {
      if (usuario === "admin" && senha === "1234") {
        resolve("Login bem-sucedido!");
      } else {
        reject("Usuário ou senha incorretos.");
      }
    }, 2000); // 2 segundos de "atraso"
});
```

CRIANDO PROMISE

```
minhaPromise("marcos", "123").then(sucesso => {
    console.log(sucesso);
}).catch(falha => {
    console.log(falha);
});
```

CRIANDO PROMISE



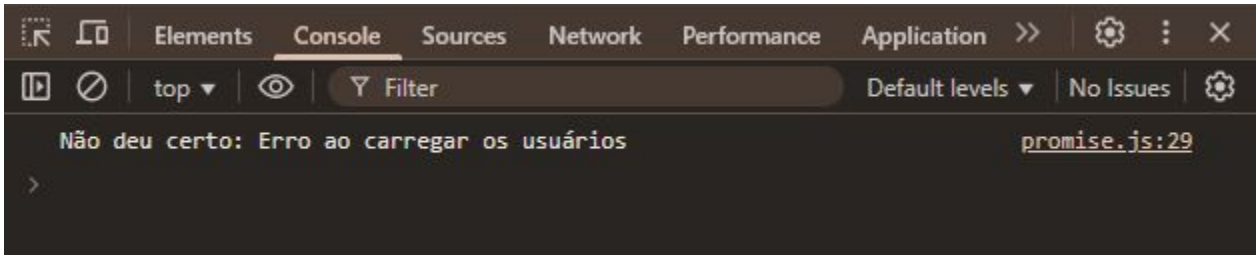
CRIANDO PROMISE

```
const promiseCarregandoUsuarios = new Promise((resolve, reject) => {
    setTimeout(() => {
        const sucesso = Math.random() * 10 >= 5;
        if(sucesso) resolve("Usuários carregdos");
        else reject("Erro ao carregar os usuários");
    }, 2000);
});
```

CRIANDO PROMISE

```
promiseCarregandoUsuarios.then(resultado => {
    console.log("Deu certo: " + resultado);
}).catch(falha => {
    console.log("Não deu certo: " + falha);
});
```

CRIANDO PROMISE



TRABALHANDO COM MÚLTIPLAS PROMISES

Quando trabalha-se com múltiplas Promises simultâneas, faz-se necessário a utilização de funções específicas para trabalhar com elas: **race()** e **all()**.

Promise.all()

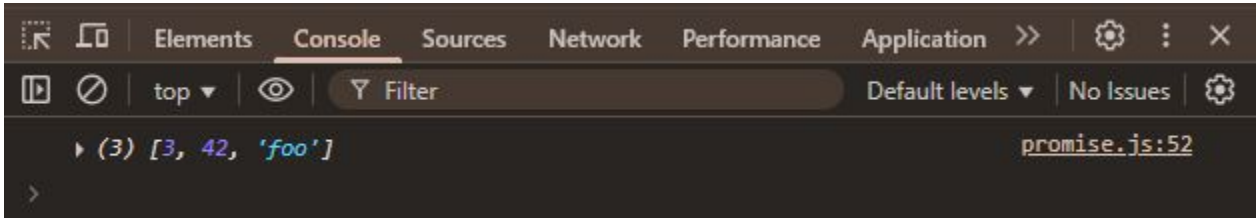
Espera todas as Promises terminarem:

```
const promise1 = Promise.resolve(3);
const promise2 = 42; // Valor não-promise é convertido
const promise3 = new Promise((resolve) => {
    setTimeout(resolve, 1000, "foo");
});

Promise.all([promise1, promise2, promise3])
    .then((valores) => {
        console.log(valores); // [3, 42, "foo"] após 1 segundo
    })
    .catch(erro => {
        console.error("Uma das promises falhou:", erro);
    });
```

Promise.all()

Espera todas as Promises terminarem:



Promise.race()

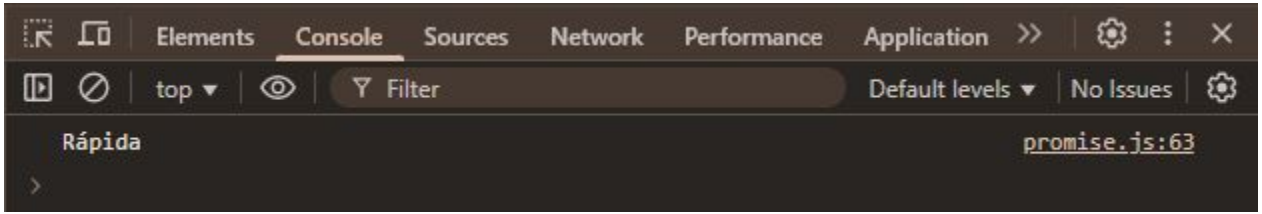
Primeira promise a ser resolvida/rejeitada

```
const promiseRapida = new Promise(resolve => setTimeout(resolve, 1000, "Rápida"));
const promiseLenta = new Promise(resolve => setTimeout(resolve, 3000, "Lenta"));

Promise.race([promiseRapida, promiseLenta])
    .then(vencedora => {
        console.log(vencedora); // "Rápida" (após 1s)
    });
```

Promise.race()

Primeira promise a ser resolvida/rejeitada



async/await

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA PARAÍBA

ASYNC/AWAIT

Recurso sintático que permite escrever código **assíncrono** de forma que se pareça e se comporte como código **síncrono**, mas sem bloquear o fluxo principal do programa.

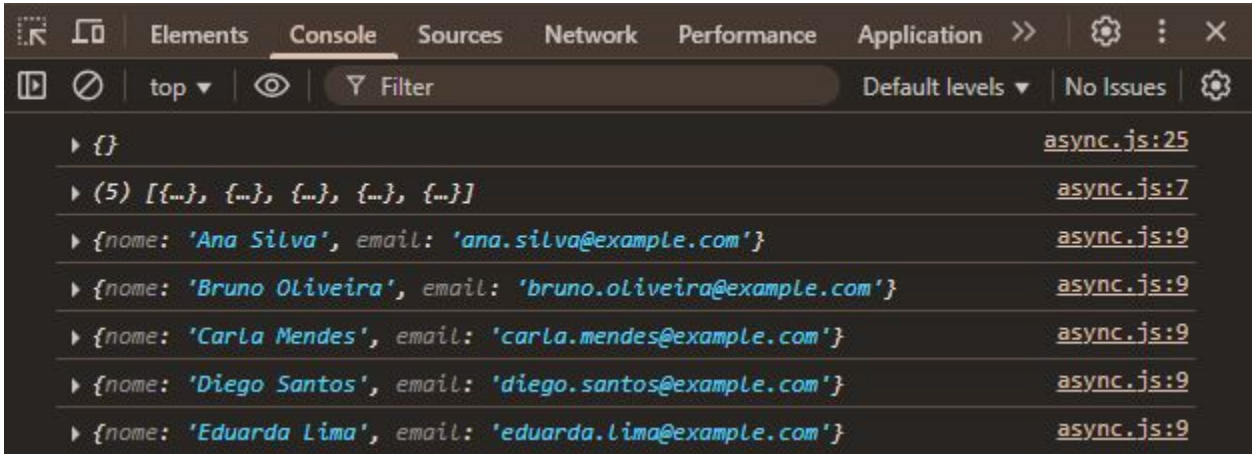
ASYNC/AWAIT

A palavra-chave **async** declara uma função que sempre retornará uma **Promise**, enquanto **await** é usada dentro dessas funções para **pausar** a execução até que uma **Promise** seja resolvida, retornando seu valor ou lançando um erro.

ASYNC/AWAIT

```
function minhaFuncaoAssincrona(user) {
    const buscaApi = fetch("https://raw.githubusercontent.com/raimundoludgerio/material-didatico/refs/heads/main/web/
    javascript/aula02/usuarios.json");
    let userFound = {}
    buscaApi.then(res => {
        const users = res.json();
        users.then(usuarios => {
            console.log(usuarios)
            usuarios.forEach(u => {
                console.log(u)
                if(u.email == user.email) {
                    userFound = u
                };
            });
        });
    });
    return userFound;
}
```

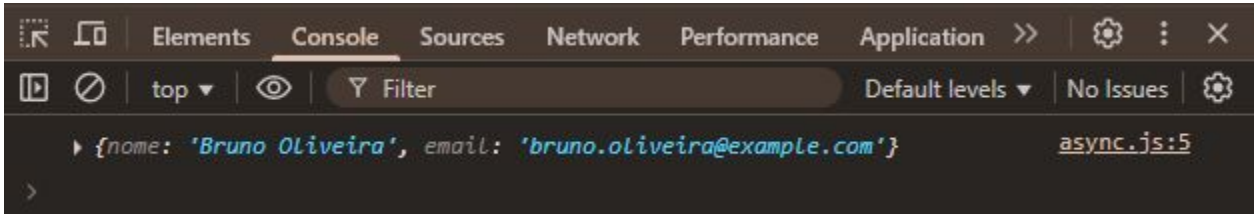
ASYNC/AWAIT


Elements
Console
Sources
Network
Performance
Application
top
Filter
Default levels
No Issues
{}
async.js:25
(5) [{…}, {…}, {…}, {…}, {…}]
async.js:7
{nome: 'Ana Silva', email: 'ana.silva@example.com'}
async.js:9
{nome: 'Bruno Oliveira', email: 'bruno.oliveira@example.com'}
async.js:9
{nome: 'Carla Mendes', email: 'carla.mendes@example.com'}
async.js:9
{nome: 'Diego Santos', email: 'diego.santos@example.com'}
async.js:9
{nome: 'Eduarda Lima', email: 'eduarda.lima@example.com'}
async.js:9

ASYNC/AWAIT

```
async function minhaFuncaoAssincrona(user) {
    const buscaApi = await fetch("https://raw.githubusercontent.com/raimundoludgerio/material-didatico/refs/heads/
    main/web/javascript/aula02/usuarios.json");
    const res = await buscaApi.json()
    res.forEach(u => {
        if(u.email === user.email) console.log(u);
    });
}
```

ASYNC/AWAIT


Elements
Console
Sources
Network
Performance
Application
top
Filter
Default levels
No Issues
{nome: 'Bruno Oliveira', email: 'bruno.oliveira@example.com'}
async.js:5

async/await - Tratamento de Erros

O tratamento de erros com **async/await** é feito usando um bloco **try...catch**, que permite capturar e gerenciar exceções de forma limpa e familiar, simplificando o tratamento de erros em operações assíncronas.

ASYNC/AWAIT - TRATAMENTO DE ERROS

```
async function essaFuncaoTemErro() {
    try {
        const buscaApi = await fetch("url_invalida");
        if (!buscaApi.ok) throw new Error("Requisição Falhou");
        const res = await buscaApi.json()
        return res;
    } catch (erro) {
        console.error("Erro capturado: " + erro)
    }
}
```

async/await - Tratamento de Erros

O tratamento de erros com **async/await** é feito usando um bloco **try...catch**, que permite capturar e gerenciar exceções de forma limpa e familiar, simplificando o tratamento de erros em operações assíncronas.

COMPONENTIZAÇÃO

COMPONENTIZAÇÃO

Componentização é a **divisão da interface** de um site em partes menores e reutilizáveis, chamadas componentes.

Por que usar componentização?

- Reutilização: o mesmo “card de produto” pode aparecer em várias páginas.
- Organização: o código fica modular, mais legível e fácil de manter.
- Escalabilidade: facilita adicionar novas funcionalidades depois.

EXEMPLO

```
<body>
    <h1>Lista de Usuários</h1>
    <div id="container"></div>
    <script src="componetes.js"></script>
</body>
```

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA PARAÍBA

EXEMPLO

```
// Exemplo de "componente" de card
function criarCardUsuario(usuario) {
  const card = document.createElement('div');
  card.classList.add('card');

  card.innerHTML = `
      <h3>${usuario.nome}</h3>
      <p><strong>Email:</strong> ${usuario.email}</p>
    `;

  return card;
}
```

EXEMPLO

```
async function carregarUsuarios() {
  const usuariosPromise = await fetch(
    "https://raw.githubusercontent.com/raimundoludgerio/material-didatico/refs/
  );
  const users = await usuariosPromise.json();
  return users
}
```

EXEMPLO

```
carregarUsuarios().then(res => {
  const container = document.getElementById('container');
  res.forEach(usuario => {
    const card = criarCardUsuario(usuario);
    container.appendChild(card);
  });
})
```

EXEMPLO


Lista de Usuários
Ana Silva
Email:
ana.silva@example.com
Bruno Oliveira
Email:
bruno.oliveira@example.com
Carla Mendes
Email:
carla.mendes@example.com
Diego Santos
Email:
diego.santos@example.com
Eduarda Lima
Email:
eduarda.lima@example.com

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA PARAÍBA

COMPONENTIZAÇÃO

- **criarCardUsuario()** é o componente, recebe um objeto e devolve um elemento HTML
- A função pode ser reutilizada várias vezes
- Em vez de escrever HTML repetido, criamos os elementos dinamicamente com JS.

Fazendo requisições a uma API pública

Consumindo dados reais

Vamos criar uma aplicação web que exibe e filtra os dados dos deputados do país consumindo a api: https://dadosabertos.camara.leg.br/swagger/api.html

Estrutura do HTML

- um campo de busca (para filtrar deputados),
- um container onde os cards serão inseridos dinamicamente,
- e uma mensagem de status (carregando, erro, etc).

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA PARAÍBA

Estilo visual

- O CSS abaixo cria um layout limpo, responsivo e com cards modernos usando grid.
- Inclui também uma paleta verde inspirada na identidade visual da Câmara.

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA PARAÍBA

JavaScript

- Buscar os dados reais da API: https://dadosabertos.camara.leg.br/api/v2/deputados
- Converter o resultado para JSON.
- Criar cards dinâmicos com nome, partido, UF e foto.
- Implementar filtro de busca no campo #busca.

OBRIGADO!

Repositório com os códigos: https://github.com/raimundoludgerio/estrutura-de-dados/tree/main/web

MDN contributors. **Introdução ao JavaScript Async**. 2025. Disponível em: https://developer.mozilla.org/pt-BR/docs/Learn_web_development/Extensions/Async_JS/Introducing#:~:text=A%20programa%C3%A7%C3%A3o%20ass%C3%ADncrona%20%C3%A9%20uma,que%20essa%20tarefa%20seja%20conclu%C3%ADda.

Contato:

raimundo.marcos@ifpb.edu.br

As Dúvidas podem ser enviadas pelo e-mail, conversar pessoalmente após a aula ou em algum horário a combinar.